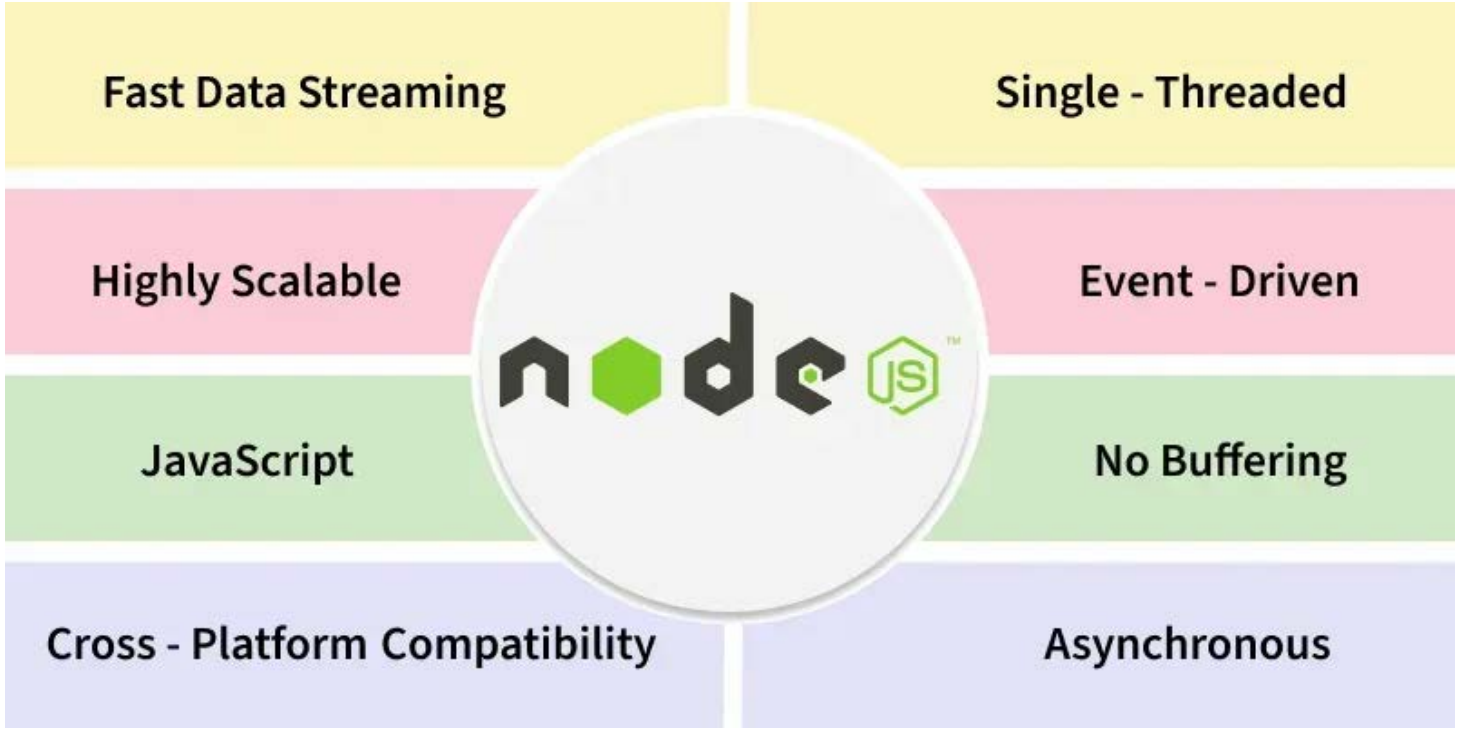
FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE 6 – NODE.JS

NODE.JS INTRODUCTION

- NodeJS is a runtime environment for executing **JavaScript** outside the browser, built on the V8 JavaScript engine.
- It enables server-side development, supports asynchronous, event-driven programming, and efficiently handles scalable network applications.



NODE.JS INTRODUCTION

- **Features of Node.js**
- **Event-Driven, Non-Blocking I/O:** Handles multiple tasks simultaneously without waiting for previous ones to complete, making it efficient for real-time applications.
- **Single-Threaded Model:** It manages numerous connections efficiently using a single thread, thanks to its event loop mechanism.
- **Rich Ecosystem:** It offers a large collection of libraries and modules through npm, simplifying development.
- **Cross-Platform:** NodeJS is cross-platform, which means that it can run on various operating systems like Windows, macOS, and Linux without modification.
- **Fast Execution:** The V8 engine compiles JavaScript into machine code before execution, which helps NodeJS achieve high performance.
- **Package Management:** npm (Node Package Manager) is a huge advantage, as it provides access to a wide range of libraries and tools that can be easily integrated into NodeJS applications.

NODE.JS INTRODUCTION

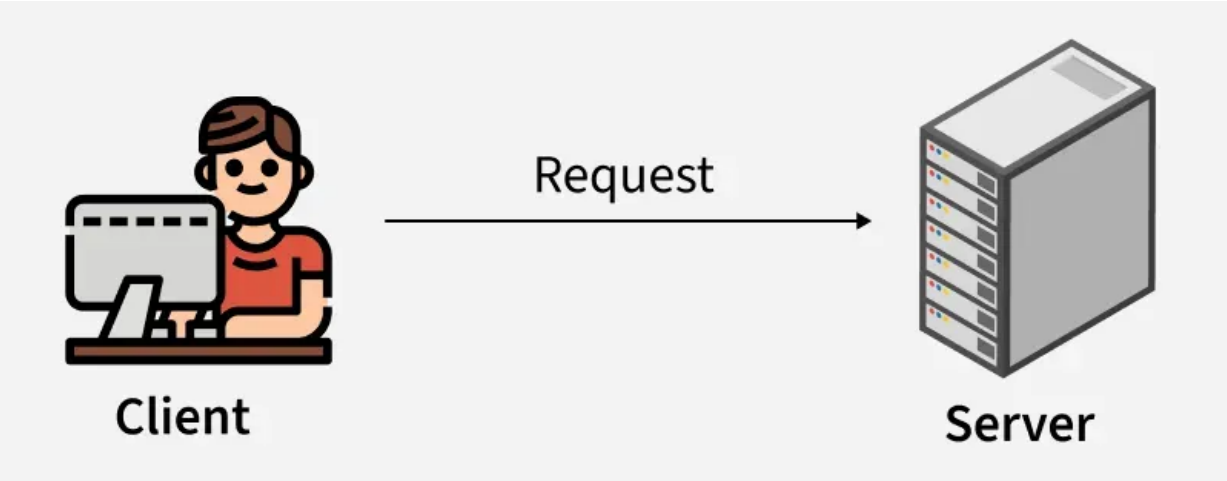
- **Benefits of Using NodeJS**
- **High Performance:** Built on Chrome's V8 engine, NodeJS compiles JavaScript to machine code, ensuring fast execution.
- **Scalability:** Designed to scale applications horizontally and vertically, efficiently managing increased workloads.
- **Unified Language:** Allows developers to use JavaScript for both client-side and server-side development, streamlining the coding process.

NODE.JS INTRODUCTION

- **Use Cases of Node.js**
- **Real-Time Applications**: Ideal for chat platforms, gaming, and collaborative tools that require instant data updates.
- **API Development**: Efficiently handles multiple simultaneous requests, making it suitable for building RESTful APIs.
- **Single-Page Applications (SPAs):** Support dynamic content loading without full page reloads, enhancing user experience.
- **Restful API and Microservices:** It easily handles microservices and API due to its lightweight, scalable, and event-driven nature.
- **Streaming Applications(Netflix)**: Node.js processes large data streams without buffering, perfect for video and audio streaming.
- **Command-line tools**: Node.js allows building a powerful CLI tool using npm libraries and cross-platform support.
- **IOT Solutions:** It's asynchronous, event-driven, which helps manage multiple IoT device connections simultaneously.

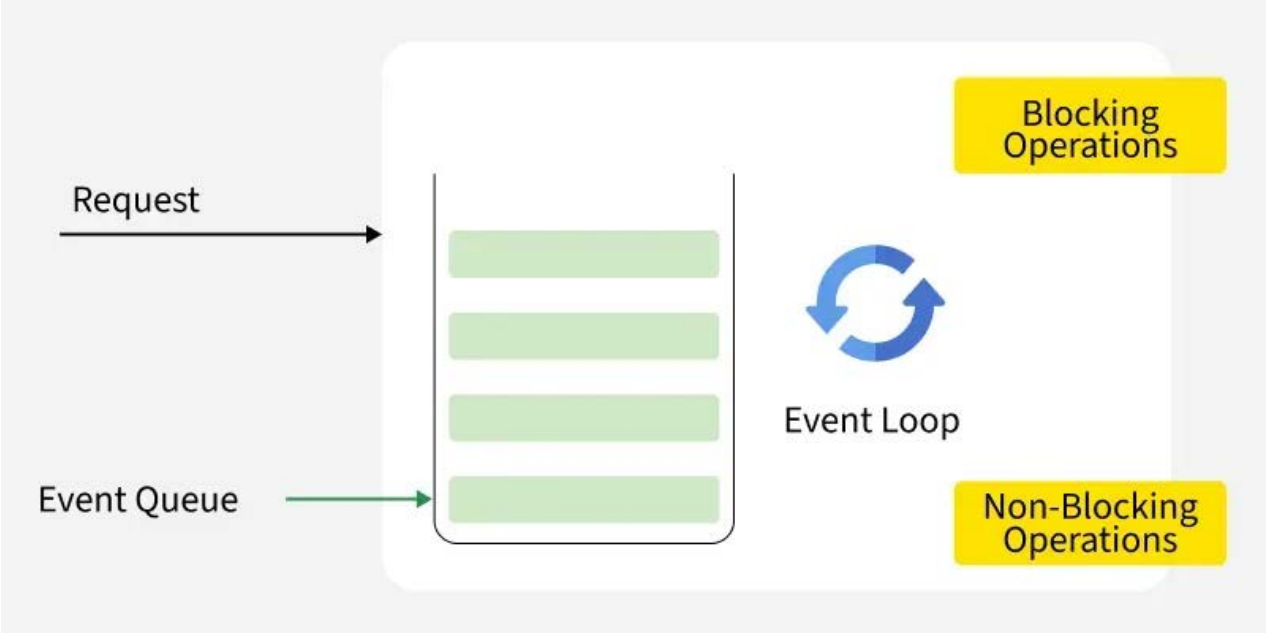
NODE.JS INTRODUCTION

- **How Node.js Handles Client Requests**
- In a Node.js environment, handling client requests efficiently is Important for building fast and scalable web applications.
- When a client interacts with a web application, it sends a request to the web server.



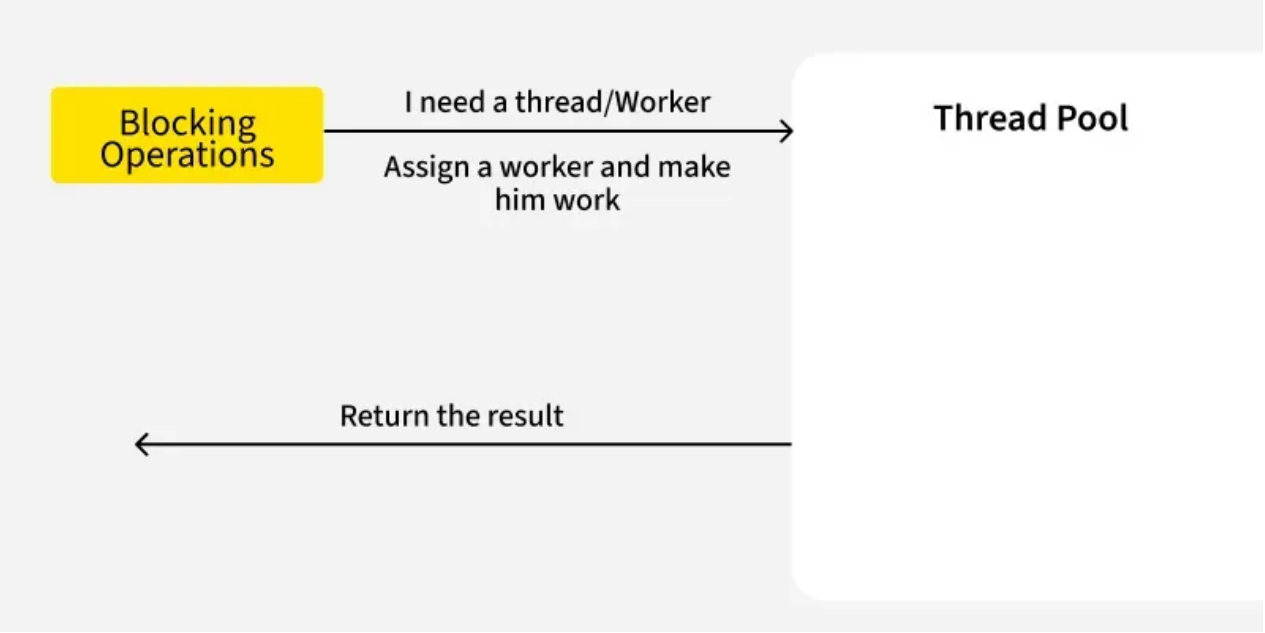
NODE.JS INTRODUCTION

- This request can either be blocking (synchronous) or non-blocking (asynchronous).
- Node.js uses an Event-Driven Architecture where all incoming requests are first placed into an Event Queue.
- An Event Loop continuously monitors this queue and processes each request accordingly.



NODE.JS INTRODUCTION

- Understanding how Node.js handles these requests using the Event Queue, Event Loop, and Thread Pool is essential to optimizing performance and avoiding delays caused by blocking operations.



NODE.JS INTRODUCTION

- **Follow Step-by-Step to send Request to Node.js Server**
- The client sends a request to the Node.js web server to interact with the web application.
- The request can be either **blocking (synchronous)** or **non-blocking (asynchronous)**. Node.js receives the request and adds it to the **Event Queue**. Requests from different users (User 1, 2, 3, etc.) are added to the Event Queue.
- Requests are picked in **(FIFO – First In, First Out)** order. If the request is **non-blocking**, it is processed immediately, and the response is sent back to the client. If the request is a **blocking operation**, it is passed to the **Thread Pool**.

NODE.JS INTRODUCTION

- The Thread Pool has a **limited number of threads.** If a thread is free, the blocking task is assigned to it.
- Once the task is completed, the thread returns the result back to the Event Loop, which then sends the response to the client.
- If all threads are busy, any new blocking requests must **wait** in a queue until a thread becomes free.
- This waiting increases the **response time** for the client. Therefore, it is always better to **avoid blocking operations** and use **non-blocking alternatives** whenever possible to maintain performance and scalability.

NODE.JS INTRODUCTION

- **Blocking or Synchronous Operation**
- In a **blocking** or **synchronous** operation, tasks are executed **one at a time** in a specific sequence.
- The program waits for the current task to complete before moving on to the next one.
- This means if a task takes time—like reading a file or making a network request—the entire program waits until that task finishes.
- While this approach is simple and easy to understand, it can slow down performance, especially when handling multiple requests or time-consuming operations.

NODE.JS INTRODUCTION

- **Non-Blocking or Asynchronous Operation**
- In a **non-blocking** or **asynchronous** operation, tasks are executed **without waiting** for the previous one to finish.
- Instead of pausing the program, long-running operations like file reading or API calls are handled in the background.
- Once the task is complete, a callback or promise is used to handle the result.
- This approach allows the program to stay responsive and handle multiple tasks at the same time, making it ideal for high-performance and real-time applications like web servers.

NODE.JS INTRODUCTION

- **How to Install Node.js on Linux using Package Manager**
- For most Linux distributions, the easiest way to install Node.js is through the default package manager.
- **Step 1: Update your system**
  - Before installing Node.js, make sure your package list is up to date.

```
sudo apt update
```

- **Step 2: Upgrade the system**
  - Once updates are installed you need to upgrade your system.

```
sudo apt upgrade
```

NODE.JS INTRODUCTION

- **Step 3: Install Node.js**
  - Ubuntu and Debian come with the apt package manager, which can be used to install Node.js. You can install Node.js directly from the official Ubuntu repositories.

```
sudo apt install nodejs
```

- **Step 4: Install npm**
  - npm is the package manager that comes bundled with Node.js. If it is not installed automatically, you can install it separately.

```
sudo apt install npm
```

- **Step 5: Verify the Installation**
  - To confirm that Node.js and npm are installed correctly, you can check their versions using:
    - Type **`node -v`** and press Enter to check the Node.js version.
    - Type **`npm -v`** and press Enter to check the npm version.
    - Both commands should return version numbers, confirming successful installation.

NODE.JS INTRODUCTION

- **Create a NodeJS Project**
- Create a new directory for your project and initialize it with npm by running.

```
mkdir node-project
cd node-project
```

- This will generate a package.json file, which is essential for managing your project dependencies.

```
npm init -y
```

NODE.JS INTRODUCTION

- **Write Your First NodeJS Application**
- Create a new file called app.js and add the following code to create a simple HTTP server:

```
const http = require('http');

const server = http.createServer((req, res) => {
    res.write('Hello World!');
    res.end();
});

server.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

NODE.JS INTRODUCTION

- **In this example**
  - The http module is imported using require('http') to allow you to create an HTTP server.
  - http.createServer creates a new server that listens for incoming HTTP requests.
  - The callback function (req, res) handles requests and responses.
  - Inside the callback, res.write('Hello, World!') sends "Hello, World!" as a response, and res.end() ends the response.
  - server.listen(3000) starts the server on port 3000. Once the server is running, it logs "Server running on port 3000" to the console.

MODULES IN NODE.JS

- In NodeJS, modules are encapsulated units of code that can be reused across different parts of an application.
- Modules help organize code into smaller, manageable pieces, promote code reusability, and facilitate better maintainability and scalability of NodeJS applications.

MODULES IN NODE.JS

- **Types of Modules:**
- **Core Modules**: Core modules are built-in modules provided by NodeJS. They offer essential functionalities such as file system operations (`fs`), HTTP server (`http`), and utilities (`util`). Core modules can be accessed using the `require()` function without specifying a path.

```
const fs = require('fs');
```

- **Third-Party Modules**: Third-party modules are created by the NodeJS community or external developers and are hosted on package registries like npm (Node Package Manager). Developers can install third-party modules using npm and include them in their applications using `require()`.

```
npm install package-name
const package = require('package-name’);
```

- **Custom Modules**: Custom modules are user-defined modules created by developers to encapsulate reusable code. Developers can create custom modules by defining functions, objects, or classes in separate files and exporting them using the `module.exports` or `exports` object.

```
// index.js
const math = require('./math');
console.log(math.add(5, 3));
```

```
// math.js
exports.add = (a, b) => a + b;
exports.subtract = (a, b) => a - b;
```

MODULES IN NODE.JS

- **Benefits of Using Modules:**
- **Encapsulation**: Modules encapsulate code, allowing developers to hide implementation details and expose only the necessary interfaces or functionalities.
- **Code Reusability**: Modules promote code reusability by enabling developers to reuse modules across different parts of an application or in multiple applications.
- **Maintainability**: Modular code is easier to maintain and refactor, as changes made to a module's implementation do not affect other parts of the application.
- **Scalability**: Modules help organize code into smaller, manageable units, making it easier to scale applications as they grow in complexity.

CORE MODULES IN NODE.JS

List of Commonly Used Node.js Core Modules

1. **fs (File System)**

- The **File System (fs) module** in Node.js is a built-in module that allows you to work with the file system on your computer.
- It provides important methods to create, read, update, delete, and rename files and directories.
- The fs.readFile() is used to read the contents of a file named **`john.txt`** asynchronously.
- If an error occurs during the file read operation, it throws an error; otherwise, it logs the file's content to the console.

```
// fs.js
const fs = require('fs');
fs.readFile('john.txt', 'utf8', (err, data) => {
    if (err) {
        throw err;
    }
    console.log('John, You have Created file system successfully');
});
```

CORE MODULES IN NODE.JS

- **fs.appendfile**
- fs.appendFile is used to add data inside file successfully. It will use fs.appendFile method to update file data . It will take three parameters file name, file data and callback function. The callback function will take parameter err . If there is any error it will throw the error on the console, if there is no error it will print the message.

```
// fsappendSync.js
const fs = require('fs');
fs.appendFile(‘john.txt', 'file added successfully',(err)=>{
    if(err)throw error;
    console.log('Data added in my file Successfully');
})
```

CORE MODULES IN NODE.JS

- **fs.appendfileSync**
- It is synchronous method from Node.js built-in module. It is used to append data to file. If the specified file will not be created fileappendSync will automatically create it.

```
const fs = require('fs');
fs.appendFileSync(‘john.txt', 'file added Successfully')
    console.log('data added in file successfully');
```

CORE MODULES IN NODE.JS

2. **http and https(HTTP/HTTPS)**

- This module allows you to create HTTP and HTTPS servers and make HTTP requests.
- You can build web servers, RESTful APIs, and clients for interacting with web services. Functions like `createServer` and `request` are part of these modules.
- The `http` module is imported using `require('http')`, and `http.createServer()` is used to create a server that takes a request (`req`) and response (`res`) object. When the server receives a request, it sends back the message: `"Hello John, this is your first http server"`. The server is instructed to listen on port `8080`, and once it starts, it logs: `"John, your server is running on port 8080"` to the console.

```
const http = require('http');
const myServer = http.createServer((req, res) => {
    res.end('Hello John, this is your first HTTP server');
});
myServer.listen(8080, () => {
    console.log('John, your server is running on port 8080');
});
```

CORE MODULES IN NODE.JS

3. **events(EventEmitter)**

- Node.js is known for its event-driven architecture, and the events module is at its core.
- It provides the EventEmitter class, allowing you to create custom events and handle them asynchronously.
- This module is the foundation for building real-time applications and custom event-based logic.

```
// EventEmitter.js
const EventEmitter = require('events');
const myEmitter = new EventEmitter();
myEmitter.on('Message', (msg) => {
    console.log(`Received: ${msg}`);
});
myEmitter.emit('Message', 'Hello John');
```

CORE MODULES IN NODE.JS

- This Node.js code uses the built-in events module to create and handle custom events. It begins by importing the EventEmitter class and creating an instance called myEmitter.
- Then, it sets up a listener using **myEmitter.on()** for an event named **'Message'.**
- Whenever this 'Message' event is triggered, the provided callback function runs and prints the received message to the console.
- Finally, the code triggers (emits) the 'Message' event using **myEmitter.emit()** and passes the string **'Hello John'** to it.

CORE MODULES IN NODE.JS

4. **path(Path)**

- The `path` module simplifies working with file and directory paths. It offers functions like `join`, `resolve`, and `basename` to manipulate and construct paths in a platform-independent way.

```
// path.js

const path = require('path');

console.log(path.basename('users/usr0298/NodeJS/coremodules/index.html'));
```

- This Node.js code uses the built-in `path` module to extract the base file name from a given path. By calling `path.basename('index.html')`, it simply returns `'index.html'` because there's no directory in the path, just the file name.

CORE MODULES IN NODE.JS

5. **util(Utilities)**

- The `util` module contains utility functions that extend JavaScript's built-in capabilities. It's particularly useful for debugging and working with objects. You'll find functions like `promisify`, `inspect`, and `format` here.

```
// util.js
const util = require('util');
const text = util.format('Hello %s', ‘John');
console.log(text);
```

- It uses Node.js's built-in `util` module, specifically the `util.format()` method, to format a string. The `%s` is a placeholder for a string value, which gets replaced by “`John`" in this case. The result is a formatted string: **`"Hello John".`**

CORE MODULES IN NODE.JS

6. **os (Operating System)**

- The `os` module provides information about the operating system on which Node.js is running. You can access details like CPU architecture, memory, and network interfaces. It's essential for writing platform-specific code.

```
// os.js
const os = require('os');
console.log(os.platform());
console.log(os.freemem());
console.log(os.arch());
```

- This Node.js code uses the built-in `os` module to get information about the operating system. It prints the platform **(like `'linux'` or `'win32'`)**, the system architecture **(like `'x64'`)**, and the amount of free system memory (in bytes).

CORE MODULES IN NODE.JS

- **crypto(Cryptography)**
- For handling cryptographic operations like encryption, decryption, and creating hashes, the `crypto` module is indispensable. It includes functions for secure data handling and authentication.

```
// crypto.js
const crypto = require('crypto');
const hash = crypto.createHash('sha256');
const result = hash.update('Hello, John').digest('hex');
console.log(result);
```

- This Node.js code uses the built-in `crypto` module to generate a SHA-256 hash of the string `'Hello, John'`. First, it imports the `crypto` module, then creates a SHA-256 hash object using `crypto.createHash('sha256')`. The `update()` method feeds the input string into the hash function, and `digest('hex')` converts the resulting hash into a hexadecimal string. Finally, `console.log(result)` prints the generated hash value to the console, which is a fixed-length encrypted representation of the input string.

BLOCKING AND NON-BLOCKING IN NODE.JS

- In NodeJS, blocking and non-blocking are two ways of writing code. Blocking code stops everything else until it's finished, while non-blocking code lets other things happen while it's waiting.
- **Blocking in NodeJS**
- When your code runs a task, it stops and waits for that task to completely finish before moving on to the next thing.
- The program waits patiently for the current operation to finish.
- No other code can run until that operation is done.

BLOCKING AND NON-BLOCKING IN NODE.JS

```
function myFunction() {
    console.log("Starting a task...");
    // Simulate a long-running task (blocking)
    let sum = 0;
    for (let i = 0; i < 1000000000; i++) { // A big loop!
        sum += i;
    }
    console.log("Task finished!");
    return sum;
}
console.log("Before the function call");
let result = myFunction();
console.log("After the function call");
console.log("Result:", result);
```

- The for loop acts like a long-running task. The program waits for it to complete.
- The "After the function call" message doesn't print until the loop is totally done.
- This makes the code run step-by-step, one thing at a time.

BLOCKING AND NON-BLOCKING IN NODE.JS

- **Non-Blocking in NodeJS**
- Non-blocking means your program can keep doing other things while a task is running in the background. It doesn't have to stop and wait for that task to finish before moving on.
- The program doesn't wait for the current task to complete.
- Other code can run while the task is working in the background.

BLOCKING AND NON-BLOCKING IN NODE.JS

```
function myFunction() {
    console.log("Starting a task...");
    // Simulate a long-running task (non-blocking) - using setTimeout
    setTimeout(() => {
        let sum = 0;
        for (let i = 0; i < 1000000000; i++) { // A big loop!
            sum += i;
        }
        console.log("Task finished!");
        console.log("Result:", sum);
    }, 0); // The 0 delay makes it asynchronous
}
console.log("Before the function call");
myFunction(); // The program doesn't wait here
console.log("After the function call");
```

- The setTimeout makes the big loop run in the background. The program doesn't stop.
- "After the function call" prints before "Task finished!" because the loop runs later.
- This lets the program do multiple things "at the same time."

BLOCKING AND NON-BLOCKING IN NODE.JS

- Let's see how blocking and non-blocking I/O work with web servers.
- **Blocking HTTP Server**
- A blocking server handles one request at a time. If it's busy reading a file for one user, other users have to wait.

```
const http = require('http');
const fs = require('fs');
http.createServer((req, res) => {
    const data = fs.readFileSync('largeFile.txt', 'utf8'); // Blocking!
    res.end(data);
}).listen(3000);

console.log("Server running at http://localhost:3000/");
```

- **fs.readFileSync** is blocking. The server stops and waits until the entire largeFile.txt is read.
- If another user tries to access the server while it's reading the file, they have to wait.

BLOCKING AND NON-BLOCKING IN NODE.JS

- **Non-Blocking HTTP Server**
- A non-blocking server can handle many requests at the same time. If it needs to read a file, it starts the read and then goes back to handling other requests. When the file is ready, the server is told about it and then it can send the data to the user.

```
const http = require('http');
const fs = require('fs');

http.createServer((req, res) => {
    fs.readFile('largeFile.txt', 'utf8', (err, data) => { // Non-blocking!
        if (err) {
            res.writeHead(500);
            res.end("Error reading file");
            return;
        }
        res.end(data);
    });
}).listen(3000);

console.log("Server running at http://localhost:3000/");
```

- fs.readFile is non-blocking, allowing the server to handle other requests while the file is being read.
- This approach is much more efficient for concurrent requests, as the server remains responsive and doesn't wait for I/O operations to complete.

EVENT LOOP IN JS

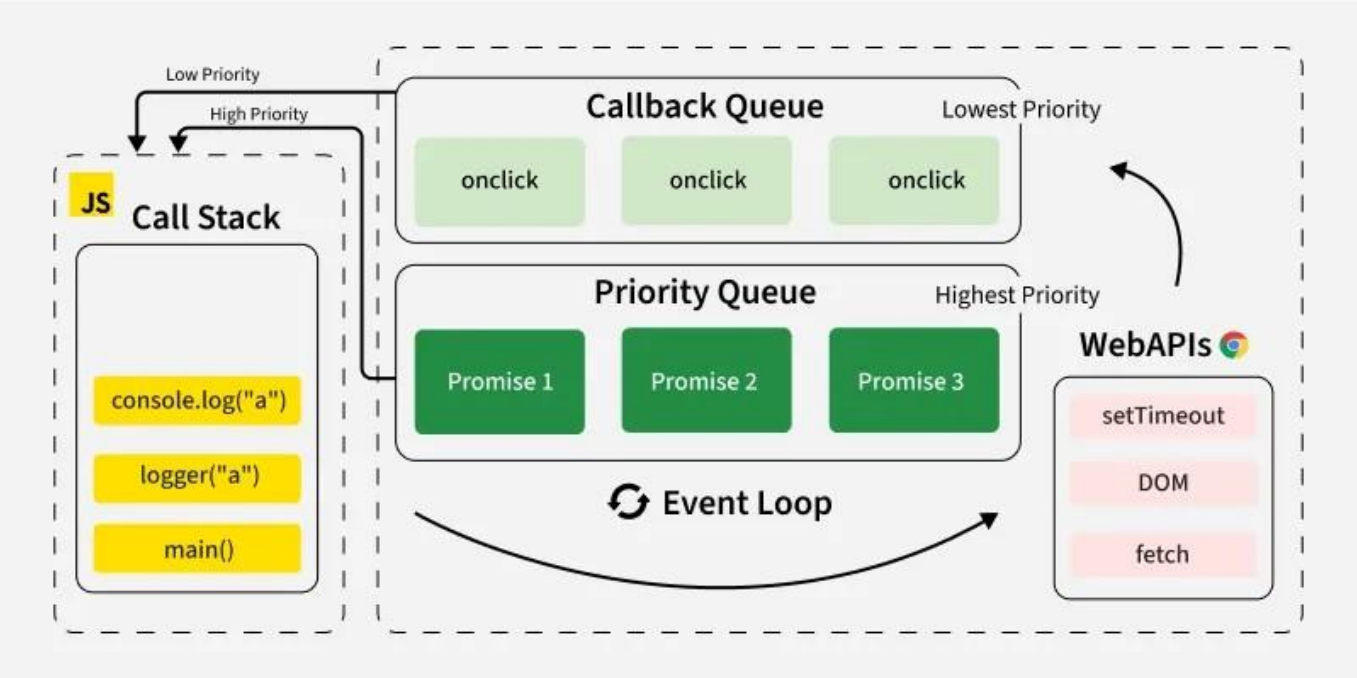
- The event loop is an important concept in JavaScript that enables asynchronous programming by handling tasks efficiently.
- Since JavaScript is single-threaded, it uses the event loop to manage the execution of multiple tasks without blocking the main thread.

```
console.log("Start");
setTimeout(() => {
    console.log("setTimeout Callback");
}, 0);
Promise.resolve().then(() => {
    console.log("Promise Resolved");
});
console.log("End");
```

- JavaScript executes code synchronously in a single thread. However, it can handle asynchronous operations such as fetching data from an API, handling user events, or setting timeouts without pausing execution. This is made possible by the event loop.

EVENT LOOP IN JS

- **How the Event Loop Works**
- The event loop continuously checks whether the call stack is empty and whether there are pending tasks in the callback queue or microtask queue.
  - **Call Stack**: JavaScript has a call stack where function execution is managed in a Last-In, First-Out (LIFO) order.
  - **Web APIs (or Background Tasks)**: These include setTimeout, setInterval, fetch, DOM events, and other non-blocking operations.
  - **Callback Queue (Task Queue)**: When an asynchronous operation is completed, its callback is pushed into the task queue.
  - **Microtask Queue**: Promises and other microtasks go into the microtask queue, which is processed before the task queue.
  - **Event Loop**: It continuously checks the call stack and, if empty, moves tasks from the queue to the stack for execution.



EVENT LOOP IN JS

- **Phases of the Event Loop**
- The event loop operates in multiple phases.
  - **Timers Phase**: Executes callbacks from setTimeout and setInterval.
  - **I/O Callbacks Phase**: Handles I/O operations like file reading, network requests, etc.
  - **Prepare Phase**: Internal phase used by Node.js.
  - **Poll Phase**: Retrieves new I/O events and executes callbacks.
  - **Check Phase**: Executes callbacks from setImmediate.
  - **Close Callbacks Phase**: Executes close event callbacks, e.g., socket.on('close').
  - **Microtasks Execution**: After each phase, the event loop processes the microtask queue before moving to the next phase.

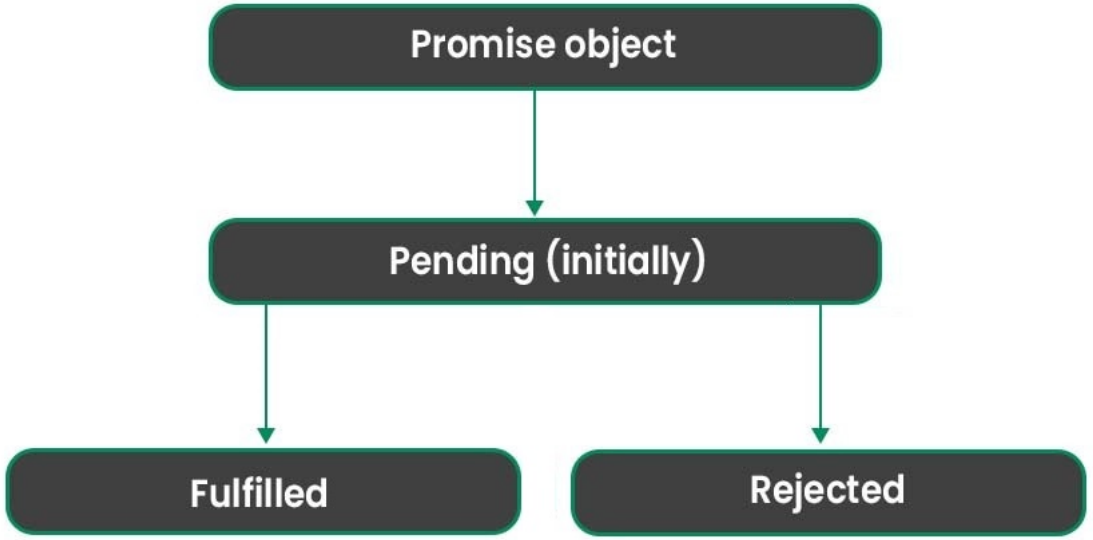
JS PROMISE CHAINING

- Promise chaining allows you to execute a series of asynchronous operations in sequence. It is a powerful feature of JavaScript promises that helps you manage multiple operations, making your code more readable and easier to maintain.
  - Allows multiple asynchronous operations to run in sequence.
  - Each then() returns a new promise, allowing further chaining.
  - Error handling is easier with a single .catch() for the entire chain.

```
function task(message, delay) {
    return new Promise((resolve) => {
        setTimeout(() => {
            console.log(message);
            resolve();
        }, delay);
    });
}
// Chaining promises
task('Task 1 completed', 1000)
    .then(() => task('Task 2 completed', 2000))
    .then(() => task('Task 3 completed', 1000));
```

JS PROMISE CHAINING

- A promise in JavaScript can be in one of three states, which determine how it behaves:
  - **Pending:** This state represents the initial state of the promise. It is neither fulfilled nor rejected
  - **Fulfilled:** This state represents that the asynchronous operation has successfully completed, and the promise has resolved with a value.
  - **Rejected:** This state represents that the asynchronous operation has failed, and the promise has rejected with a reason (error).



JS PROMISE CHAINING

- **Error Handling in Chaining**

```
Promise.resolve(5)
.then((num) => {
    console.log(`Value: ${num}`);
    throw new Error("Something went wrong!");
})
.then((num) => {
    console.log(`This won't run`);
})
.catch((error) => {
    console.error(`Error: ${error.message}`);
});
```

JS PROMISE CHAINING

```
function fetchUser(userId) {
    return Promise.resolve({ id: userId, name: “John" });
}

function fetchOrders(user) {
    return Promise.resolve([{ orderId: 1, userId: user.id }]);
}

fetchUser(101)
    .then((user) => {
        console.log(`User: ${user.name}`);
        return fetchOrders(user);
    })
    .then((orders) => {
        console.log(`Orders: ${orders.length}`);
    })
    .catch((error) => console.error(error));
```

JS PROMISE CHAINING

- **Why Use Promise Chaining?**
  - **Improved Readability:** Makes code cleaner and easier to understand compared to deeply nested callbacks.
  - **Error Handling:** Centralized error management with a single .catch() for the entire chain.
  - **Sequential Execution:** Ensures tasks are executed one after the other, passing results along.
  - **Simplifies Dependencies:** Handles tasks that depend on the output of previous asynchronous operations.
  - **Enhanced Debugging:** Stack traces in Promise chains are easier to follow compared to callback chains.

NODE.JS THIRD PARTY MODULES

- Third-party modules are external libraries maintained and built by the developer community.
- Unlike core modules that are added with Node.js, these modules must be installed separately using npm.
- **Installing and Using Third-Party Modules**
- Node.js uses **npm**, the world's largest software registry, to manage third-party packages. To install A module is straightforward; you just need to give the command **`npm install express`**.
- This command installs all the popular express web framework and adds it to the project's node_modules directory. To use the installed module you just simply need to give the **`require`** command .

```
const express = require('express');
const app = express();
app.get('/', (req, res) => {
  res.send('Hello John!');
});
app.listen(3000, () => console.log('Server running on port 3000'));
```

NODE.JS THIRD PARTY MODULES

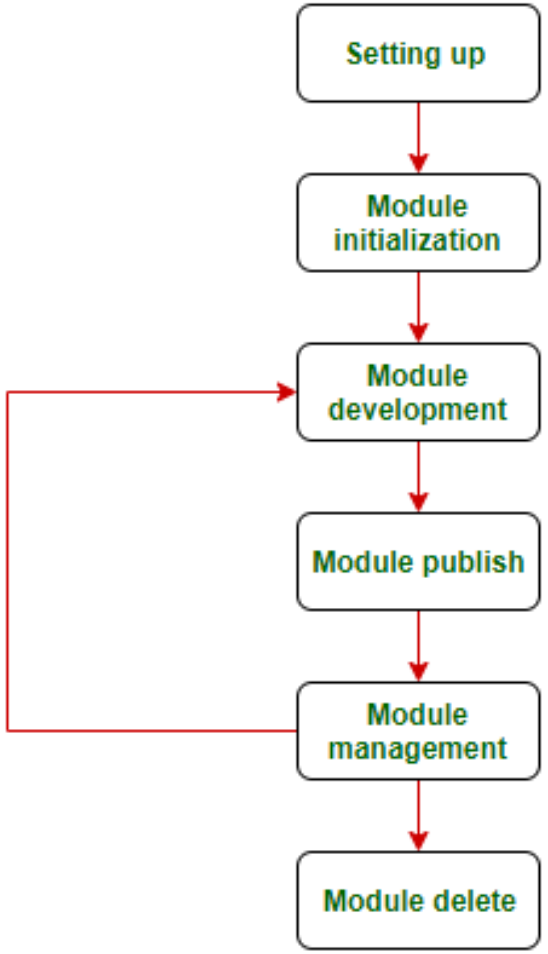
- Here are some widely adopted third-party modules and their use cases:

Module	Purpose
express	fast, minimalist web framework for building APIs and web apps
mongoose	MongoDB ODM for schema-based data modeling
axios	Promise-based HTTP client for server and browser
lodash	Utility functions for working with arrays, objects etc
dotenv	Loads environment variables from a .env file
jsonwebtoken	Implementation of JSON Web Tokens (JWT) for authentication

STEPS TO CREATE AND PUBLISH NPM PACKAGES

- The life-cycle of an npm package:

1. **Setup a Project:** Setting up a project is required before doing anything.
   - Install Node.js.
   - Create and login your npm account.
2. **Initializing a module:** To initialize a module, Go to the terminal/command-line and type npm init and answer the prompts.



STEPS TO CREATE AND PUBLISH NPM PACKAGES

- In the **version** prompt, set it to **0.0.0**. It initializes the module.
- In the main prompt, choose the entry point of the module. Note that the entry point is 'src/index.js' which is considered a standard practice these days to put your code in an 'src' directory.
- In the test command prompt, simply press Enter.
- In the git repository prompt, you can fill the URL of the git repository where the package will be hosted.
- Fill in the keywords, author, and license or you can press 'Enter' and modify them later in the package.json.
- Add **"type": "module"** in the package.json file in order to use the latest import/export ES6 feature.
- Include a **README.md** file in the project for potential downloaders to see. This will appear on the homepage of your module.

STEPS TO CREATE AND PUBLISH NPM PACKAGES

3. **Building a module:** This phase is the coding phase.

```
export const sumFns = {
  add : function addTwoNums (num1, num2 ) {
    return (num1 + num2) ;
  }
}
```

4. **Publishing a module:** After completion of the coding module, publish the npm package. To publish the package, there is one thing to keep in mind - if your package name already exists in the npm registry, you won't be able to publish it. To check if your package name is usable or not, go to the command line and type: `npm search packagename`. If your module name already exists, go to the package.json file of the npm module project and change the module name to something else. After checking the name availability, go to command-line/terminal and type: `npm publish`.

STEPS TO CREATE AND PUBLISH NPM PACKAGES

5. **Updating and managing versions:** If a software is being developed, it is obvious that it has versions. Versions are a result of bug fixes, minor improvements, major improvements, and major releases. To cater to versioning, NPM provides us with the following functionality.

- **Versioning and publishing code:** NPM allows us to version our module on the basis of semantic-versioning. There are three types of version bumps that we can do, ie, patch, minor, and major.
- **What happens when the user has an older version of the module?** When an npm module is re-published (updated), the users just have to run `npm install <packagename>` again to get the latest version.

DIFFERENCE BETWEEN --SAVE AND --SAVE-DEV IN NODEJS

- In NodeJS, when you install packages using npm (Node Package Manager), you often need to decide whether to install them as a dependency or devDependency.
- This is where the flags --save and --save-dev come into play. These flags control where the installed packages are placed in the package.json file and whether they are required in production or development environments.

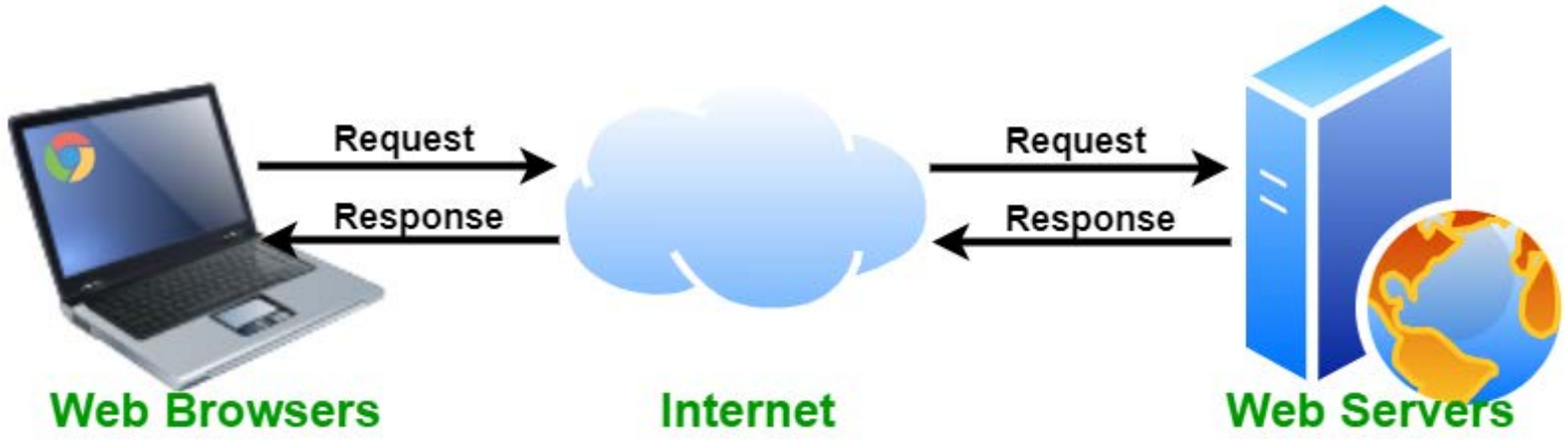
Features	**--save**	**--save-dev**
Purpose	Adds the package to dependencies in package.json, indicating it is required in production	Adds the package to devDependencies in package.json, indicating it is only needed during development
When It's Used	For packages essential for running the application in production	For packages needed only during development (e.g., testing frameworks, build tools)
Location in package.json	Added under the "dependencies" section.	Added under the "devDependencies" section.
Effect on node_modules Folder	It increases in size as production dependencies are installed.	Also increases its size, but these dependencies are ignored in production.
Production Deployment	Installed when deploying or running the app in production	Not installed in production, reducing the size of production dependencies
Examples of Packages	express, mongoose, cors, jsonwebtoken	jest, mocha, chai, eslint, webpack
Default in npm 5 and Later	By default, npm adds packages to dependencies even without the --save flag	Must explicitly use --save-dev to add to devDependencies

DIFFERENCE BETWEEN --SAVE AND --SAVE-DEV IN NODEJS

- **When to Use --save?**
  - Use npm install package (default --save) when:
  - The package is needed in production.
  - It is part of the core application (e.g., frameworks, database clients, authentication tools).
  - You want it installed when running the project in any environment.
  - **Examples**: express, mongoose, jsonwebtoken, cors
- **When to Use --save-dev?**
  - Use npm install package --save-dev when:
  - The package is only needed for development (e.g., testing, debugging, building).
  - It should not be included in production to keep the build lightweight.
  - You need linters, testing libraries, or build tools.
  - **Examples:** jest, eslint, prettier, webpack

WEB SERVER

- A web server is a system either software, hardware, or both that stores, processes, and delivers web content to users over the Internet using the HTTP or HTTPS protocol.
- An HTTP server is a specific type of web server which is responsible for handling HTTP requests and responses.
- **How Does a Web Server Work?**
- When a user accesses a website by entering a URL in their web browser, the browser sends an HTTP request to the web server hosting the website. The web server processes this request and returns the necessary resources to display the page on the user's browser.

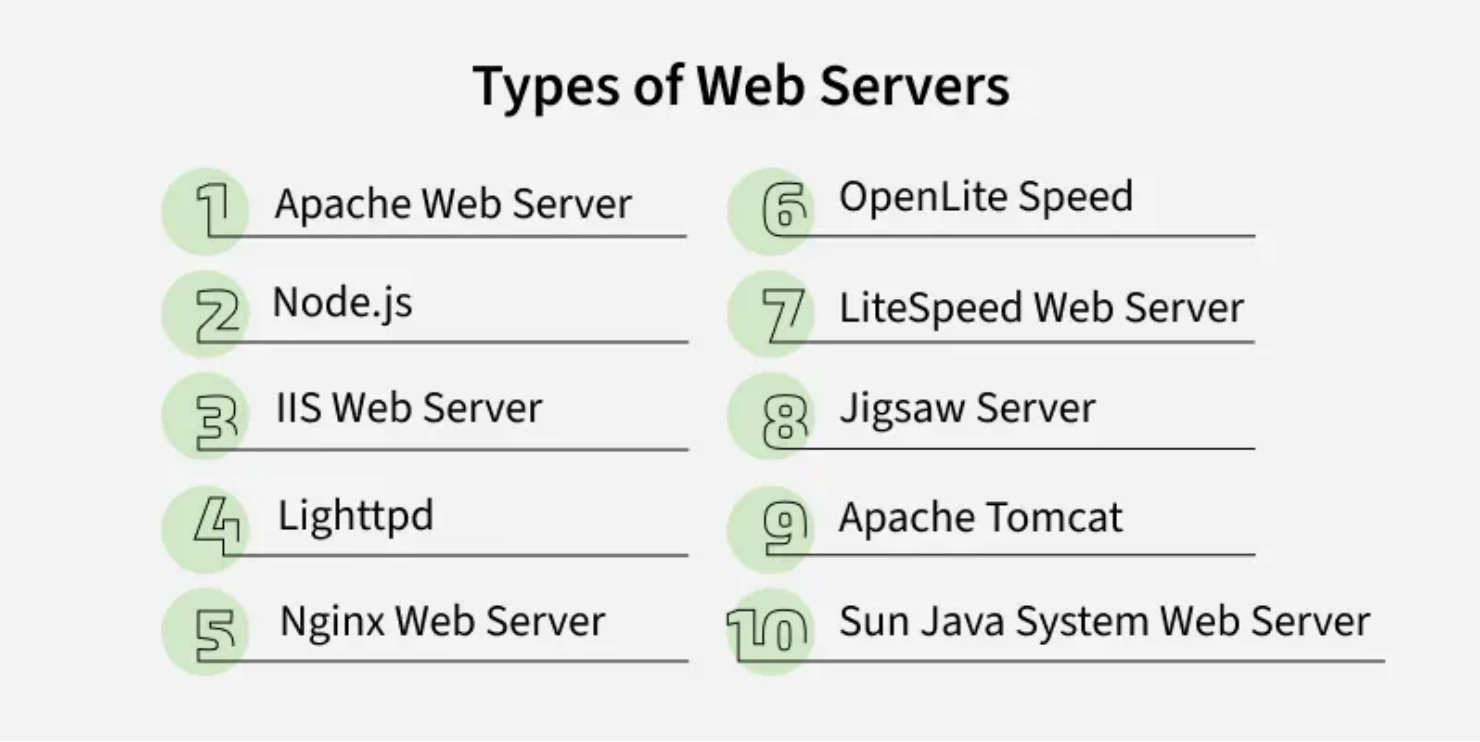


WEB SERVER

- **Here is a simplified version of how the process works**
  - **Client Request:** In the web browser(https://www.example.com/) the user enters a URL.
  - **DNS Resolution:** To get the IP address of the requested domain, the browser contacts a Domain Name System (DNS) server.
  - **Connecting to the Web Server:** Using the obtained IP address the browser establishes a connection with the web server.
  - **Processing Request:** The web server receives the request and processes it.
  - **Serving the Response:** The requested files(HTML, CSS, JavaScript, images) are sent back to the client's browser by the web server.
  - **Rendering the Web Page:** Based on the received data the browser displays the web page to the user.

TYPES OF WEB SERVERS

- Web servers can be categorized based on their functionality, usage, and implementation. Below are some of the most common types.



TYPES OF WEB SERVERS

- Choosing the right web server depends on what you need for your website or application:
  - **Use Apache**: If you want a reliable and customizable web server that works on almost any system. It's great for general websites and supports many features.
  - **Use Nginx:** If your website gets a lot of visitors and you need a fast and efficient server that can handle high traffic smoothly.
  - **Use IIS**: If you are using Windows-based applications and need a server that works well with Microsoft technologies like ASP.NET.
  - **Use LiteSpeed**: If you want a faster and more secure alternative to Apache, especially for WordPress or other PHP-based websites.
  - **Use Apache Tomcat:** If your website or app is built with Java and you need a server that supports Java Servlets and JSP (Java Server Pages).
  - **Use NodeJS**: If you're building real-time applications, such as chat apps or online games, and want to use JavaScript for both frontend and backend.
  - **Use Lighttpd**: If you need a lightweight and fast server that works well on systems with low memory or limited resources.
  - **Use OpenLiteSpeed:** If you want a free version of LiteSpeed that still offers great speed and performance.
  - **Use Jigsaw**: If you are a developer or researcher testing new web technologies and standards.
  - **Use Sun Java System Web Server:** If you are working with older Java applications, but note that this server is no longer supported.